```
        node.parent.left ← null
    else
        node.parent.right ← null
    return
true
#11
```

#1 Method `remove` takes a treap instance and the key to be removed; the method returns true if the key was removed, or false if it couldn't be found. It also has side effects on the treap passed as argument.
#2 Search for the key in the treap.
#3 If `search` returned `null`, then there is no such key stored, and hence it can't be deleted.
#4 If the treap only contained one node, then removing it will just leave an empty tree; we can check that by either testing if the treap's size is 1, or equivalently if node is both a leaf and the root.
#5 Otherwise, we need to push down this node all the way to the leaves level.
#6 Check which of `node`'s two children should replace it: it will have at least one child (because it's not a leaf), and if it has both children, we will need to choose the one with the highest priority (i.e. lowest value, in our example, because we are implementing a min-treap).
#7 If we chose the left child, we need a right rotation.
#8 Otherwise, we need a left rotation.
#9 We have to be careful in case node was the root: if that's the case, we need to update `treap`'s property. This check can be true only on the first iteration of the cycle, so it might make sense, if performance is crucial, to break the cycle down and handle the first iteration separately.
#10 After exiting the `while` loop, node is now a leaf, and certainly not the root anymore: we can just disconnect it by nulling the